

# 1 共享内存通信算子优化手册

## 1.1 引言

在鲲鹏 920F 平台做大模型推理部署时，同样会遇到模型参数量太大，高带宽内存装不下，或单 CPU 算力无法进一步提升吞吐等问题，我们用到了多种并行策略如数据并行（DP）、张量并行（TP）、专家并行（EP）、流水线并行（PP）来解决此问题。以 128P 冗余专家的部署形态为例，我们将每个 Socket 作为一个 DP，在 DP 内进行 TP-8 切分，即模型参数分摊到 8 个 NUMA 上，在计算过程中自然会使用 Allreduce、Allgather 等节点内通信算子进行计算结果的规约、拼接。可采用 MPI 标准集合通信接口或 MPI-3 标准提供的共享内存窗口完成节点内通信需求，但受以下几点限制，其通信性能难以保障。

### 1.1.1 MPI 标准集合通信接口缺陷

#### （1）标准集合通信接口功能单一

MPI 标准语义集合通信接口均以一维连续数组的数据传输为主，不支持 AI 场景下矩阵形式的数据搬运，如果切分导致 BUFFER 内存不连续，需要补充 PACK、UNPACK 的拷贝逻辑才可实现算子功能，制约算子性能。

#### （2）标准集合通信实现多线程并发支持不足

标准 MPI 集合通信接口的共享内存实现未使能多线程，且内部使用粗粒度锁保护共享资源，对多线程场景不够友好，无法真正发挥多核优势。

#### （3）标准集合通信实现存在冗余拷贝

使用 MPI 标准集合通信接口时，MPI runtime 层的封装将共享内存作为中间 BUFFER 使用，在实际的通信过程中，发送方进程将数据写入共享内存后，再由接收侧进程从共享内存读取数据，产生了一次冗余的用户态拷贝。

### 1.1.2 MPI\_SHARE\_MEMORY\_WINDOW 的局限

MPI\_SHARE\_MEMORY\_WINDOW 是 MPI 提供的节点内共享内存通信机制，允许同一节点的进程映射访问同一块物理内存，避免传统消息传递的数据拷贝，使用时由

同通信域进程协同创建共享窗口、获取本地映射地址后直接读写，需配合同步机制保障数据一致性，使用完毕后协同销毁窗口释放资源。但受限于 MPI 标准的设计和实现约束，其在实际高性能计算场景中仍存在诸多影响性能与灵活性的局限。

---

MPI 共享内存窗口使用示例

---

```
1 MPI_Win win;
2 int *shared_data;
3 MPI_Aint size = sizeof(int);           // 每个进程共享 1 个 int
4 MPI_Win_allocate_shared(size, sizeof(int), MPI_INFO_NULL, MPI_COMM_WORLD,
    &shared_data, &win);                   // 申请共享内存窗口
5 MPI_Win_fence(0, win);                  // 同步确保窗口创建完成
6 if (rank == 0) { *shared_data = 0; }    // 根进程写入数据 (Store)
7 MPI_Win_fence(0, win);                  // 同步确保写入完成
8 int local_val = *shared_data;           // 所有进程读取数据 (Load)
```

---

(1) 内存分配与映射接口的设计局限

MPI 将内存申请和映射集成在一个接口，想实现跨进程的 load/store 只能使用该接口申请出的内存，无法对业务中申请的计算 BUFFER 进行灵活映射，导致通信前需要将数据拷贝至通信 BUFFER 影响性能。此外，MPI 的内存分配都集中在 rank0，导致通信过程中，单个 numa 上的读写负载较大，无法通过拓扑感知等手段设计高性能通信算法。

(2) 零拷贝内存的使用限制

MPI 共享内存不支持无同步保护的自由读写，数据一致性必须通过 MPI\_Win\_fence、MPI\_Win\_lock 等显式同步接口保障。所有远程内存访问需置于同步区间内才能保证正确性，但冗余、非必要的同步操作会引入额外等待开销，导致通信时延上升。

1.2 典型 SHM 算子介绍及对应的并行策略场景

1.2.1 Allgather

对于 N 个进程，rank 0, 1, 2, ..., N-1，每个进程 i 持有一段数据 sendbuf\_i，Allgather 会把所有进程的数据 sendbuf\_0, sendbuf\_1, ..., sendbuf\_N-1 按进程

号顺序拼接，最终，每个进程都拥有一致的完整拼接结果，通信语义如图 1-1 所示。

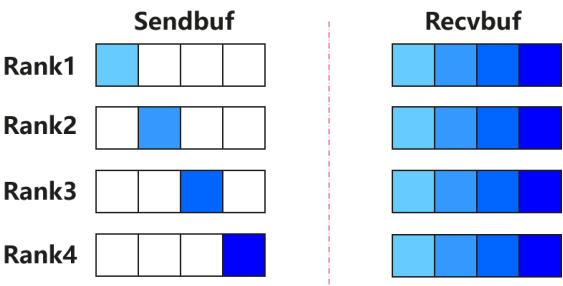


图 1-1 Allgather 算子输入输出示意图

该算子是 TP 场景下的常用算子，例如在 MLA 阶段的低秩压缩中，对投影矩阵按列切分，每个进程仅计算部分结果，此时通过 Allgather 收集所有进程的结果，得到完整的投影输出。

1.2.2 Allreduce

对于  $N$  个进程， $\text{rank } 0, 1, 2, \dots, N-1$ ，每个进程  $i$  持有一段输入向量  $\text{sendbuf}_i$ ，Allreduce 会对所有进程执行同一个规约操作，包括求和、求最值等，最终所有进程获得相同的规约结果，通信语义如图 1-2 所示。

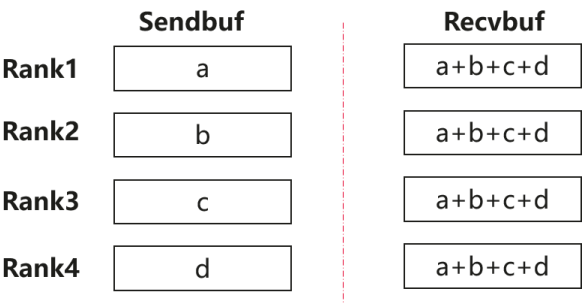


图 1-2 Allreduce 算子输入输出示意图

Allreduce 规约操作在 TP 并行的推理流程中更为常见，例如 MLA 对不同 head 的 attention 计算结果的求和，以及 MoE 阶段对不同共享专家计算结果的求和。

1.2.3 Alltoall

对于  $N$  个进程， $\text{rank } 0, 1, 2, \dots, N-1$ ，每个进程  $i$  持有一段输入向量  $\text{sendbuf}_i$ ，Alltoall 会对所有进程持有的数据进行全局交换，通信语义如

图 1-3 所示。

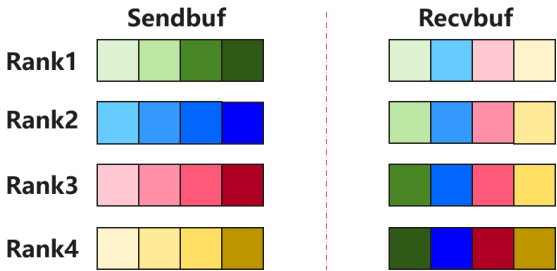


图 1-3 Alltoall 算子输入输出示意图

为了实现 MLA 阶段 KV Cache 在内存的分布式存储并提高 attention 计算的计算访存比，我们引入 Alltoall 对 MLA 阶段的激活值矩阵进行转置，将 MLA 阶段 attention 计算的 TP 切分从 head 维度转为 batch 维度。

1.3 SHM 算子原理和设计思路

1.3.1 零拷贝内存

节点内进程间通信因为虚拟地址隔离，需要通过中间 shm buffer 拷贝两次，在本方案中采用内存 attach 能力，将一进程中的一段 VA 的页映射拷贝到另一进程中的一段 VA 中，兼容 PMD 大页和 PTE 小页映射。attach 后的目的地址访问权限和源地址保持一致，完成 attach 后进行带外地址信息交换，即可拿到对端地址空间在本进程中的虚拟地址映射，从而直接进行数据搬运，减少冗余的拷贝次数。

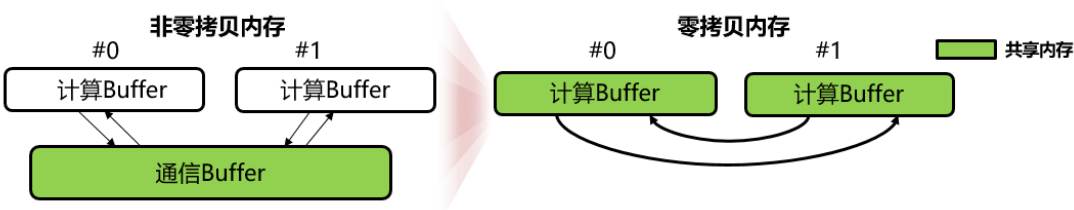


图 1-1 零拷贝内存示意图

1.3.2 多线程提升访存性能

单核一次只能有有限个未完成的访存请求（outstanding loads/stores）。内存延迟越高、允许请求越少，单核带宽越低。利用线程级并行，同时发射更多访存请求，充分利用多通道、多 bank 并行，把内存总线带宽跑满。表 1-1 展示

了 TP8 场景下，与业务中相同的包大小（输入 64\*528，输出 64\*4224），在不同线程数情况下的 Batch\_Allgather 单算子实测性能。

表 1-1 不同线程情况下 Batch\_Allgather 算子性能

| 线程数    | 1     | 2    | 4     | 8    | 16    | 32    |
|--------|-------|------|-------|------|-------|-------|
| 时延(us) | 56.67 | 32.9 | 29.78 | 19.5 | 15.48 | 14.56 |

1.3.3 拓扑感知

拓扑感知可以让数据流优先走高带宽、低延迟的链路，拥有全局拓扑信息后可以调整负载，最大化利用每条链路的流量。在单个 CPU 内部也可采用同样的思路，比如 Die 间 MCM IO 受封装引脚/PHY/SerDes 限制，带宽密度、峰值带宽、延迟均远弱于片上 Ring。针对 920F 芯片不同总线的 LMBench 实测带宽如表 1-2 所示。因此，在设计通信算法时，需将数据流量尽量留在 Die 内，仅跨 Die 数据才走 MCM，最大化利用片上高带宽资源，将带宽受限链路上的数据降至最小。

表 1-2 920F 不同拓扑距离下的 LMBench 实测带宽

|                           |     |
|---------------------------|-----|
| HBM 带宽-NUMA 内读写 (GB/s)    | 400 |
| HBM 带宽-跨 NUMA 读写 (GB/s)   | 250 |
| HBM 带宽-跨 Die 读单向 (GB/s)   | 170 |
| HBM 带宽-跨 Die 写单向 (GB/s)   | 150 |
| HBM 带宽-跨 Socket 单向 (GB/s) | 120 |

1.4 典型 SHM 算子关键技术

1.4.1 Allreduce

(1) 算法实现

在 DeepSeek 分布式推理方案中的 TP 并行部分，需要进行 8 进程 allreduce，结合鲲鹏硬件特性，我们设计如下的 allreduce 方案。

首先，在 allreduce 通信前，每个进程都持有长度为 N 的输入。为了尽可能降低跨 Die 数据，我们将 8 个进程两两配对分组来处理对端跨 Die 通信，该算法能使跨 Die 数据量达到理论下界。数据排布如下图：

初始数据排布:

|        |
|--------|
| Rank 0 |
| Rank 1 |
| Rank 2 |
| Rank 3 |
| Rank 4 |
| Rank 5 |
| Rank 6 |
| Rank 7 |

$n$

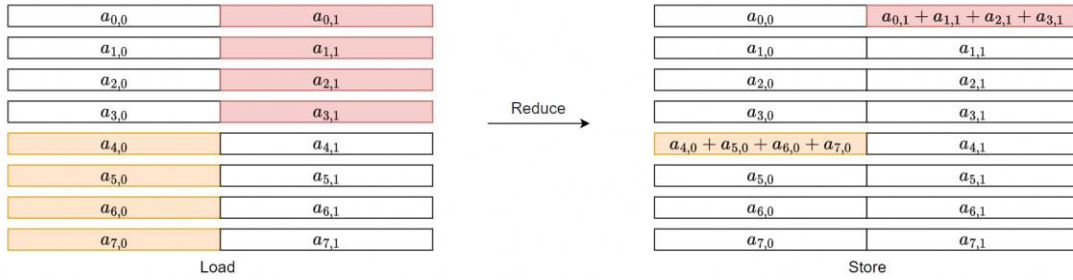
进程通信区域划分:

|                  |                  |                  |                  |
|------------------|------------------|------------------|------------------|
|                  |                  |                  |                  |
|                  |                  |                  |                  |
|                  |                  |                  |                  |
| Rank 0,4<br>通信区域 | Rank 1,5<br>通信区域 | Rank 2,6<br>通信区域 | Rank 3,7<br>通信区域 |
|                  |                  |                  |                  |
|                  |                  |                  |                  |
|                  |                  |                  |                  |
|                  |                  |                  |                  |

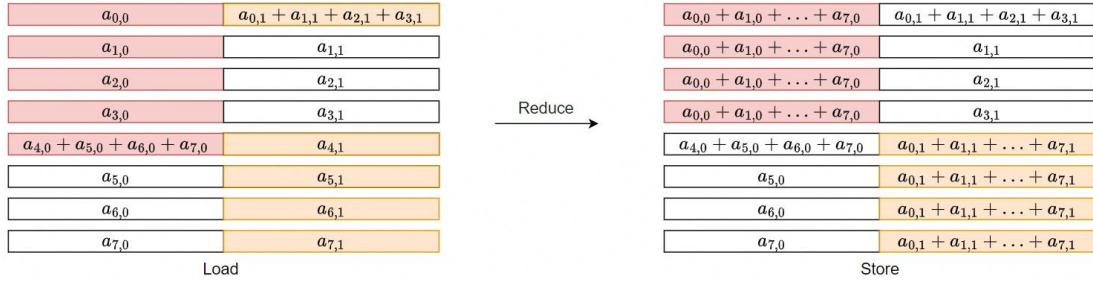
$\frac{n}{4} \quad \frac{n}{4} \quad \frac{n}{4} \quad \frac{n}{4}$

对于每一对进程，通信流程分为 3 个 Phase，相邻 Phase 间需要点对点进程同步。红色表示第 1 个进程操作的数据区域，黄色表示第 2 个进程。具体如下：

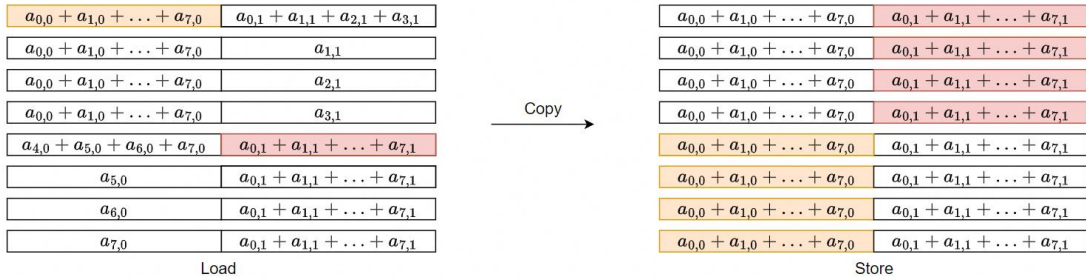
Phase 1: Die 内 Reduce



Phase 2: 跨 Die Reduce



Phase 3: 跨 Die Copy



(2) 算子性能

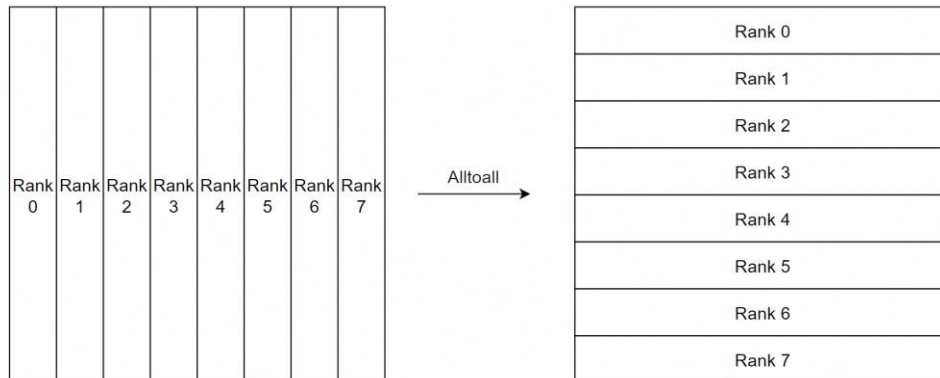
结合 1.3.3 节理论带宽数据和 allreduce 各阶段的通信数据量，可以推算得到：Phase 1 和 2 的性能瓶颈在跨 NUMA 访存，Phase 3 的瓶颈则在跨 Die 访存。在下表中，我们给出不同 size 下 allreduce 算子的性能数据：

|             |                       |        |        |        |
|-------------|-----------------------|--------|--------|--------|
| 输入 size(KB) |                       | 896    | 1792   | 3584   |
| 时延(us)      |                       | 24.27  | 34.88  | 66.69  |
| 算子拆解时延(us)  | Phase 1: Die 内 Reduce | 3.34   | 5.89   | 11.68  |
|             | Phase 2: 跨 Die Reduce | 5.10   | 10.75  | 23.62  |
|             | Phase 3: 跨 Die Copy   | 3.21   | 6.38   | 17.46  |
|             | 同步                    | 12.63  | 11.87  | 13.93  |
| 带宽(GB/s)    | Phase 1 瓶颈: 跨 NUMA 读写 | 167.57 | 190.01 | 191.78 |
|             | Phase 2 瓶颈: 跨 NUMA 读写 | 197.66 | 187.60 | 170.73 |
|             | Phase 3 瓶颈: 跨 Die 读   | 139.68 | 140.52 | 102.63 |
| 带宽利用率       | Phase 1 瓶颈: 跨 NUMA 读写 | 67.0%  | 76.0%  | 76.7%  |
|             | Phase 2 瓶颈: 跨 NUMA 读写 | 79.1%  | 75.0%  | 68.3%  |
|             | Phase 3 瓶颈: 跨 Die 读   | 82.2%  | 82.7%  | 60.4%  |

### 1.4.2 Alltoall

#### (1) 算法实现

Alltoall 算子用于 DeepSeek 分布式推理方案中 TP 切分维度的转换。具体地，以 TP=8 为例，对形状为(w, h)的完整激活值矩阵的 h 维度进行 TP 并行，则各进程持有(w, h/8)的数据，通过 Alltoall 进行二维转置，矩阵形状变换为(w/8, h)，具体数据排布和 Alltoall 形式如下图：



在通信算法上，由于 alltoall 不存在 reduce 操作，我们采用基于 shm 直接读取远端进程内存数据的方式，就能达到跨 Die 数据量的下界，同时实现跨 Die 和跨 NUMA 通信的相互隐藏。

#### (3) 算子性能

结合 1.3.3 节理论带宽数据和 alltoall 的通信数据量，可以发现 alltoall 的性能瓶颈在跨 Die 访存。在下表中，我们给出不同 size 下 alltoall 算子的性能数据：

|              |               |        |        |        |
|--------------|---------------|--------|--------|--------|
| 输入 size (KB) |               | 1152   | 2304   | 4608   |
| 单算子时延 (us)   |               | 26.78  | 43.09  | 81.10  |
| 算子拆解时延 (us)  | Socket 内 Copy | 22.76  | 38.68  | 74.92  |
|              | 同步            | 4.02   | 4.41   | 6.18   |
| 带宽 (GB/s)    | 瓶颈: 跨 Die 单向读 | 101.25 | 119.12 | 123.01 |
| 带宽利用率        | 瓶颈: 跨 Die 单向读 | 59.6%  | 70.1%  | 72.4%  |

### 1.4.3 Allgather

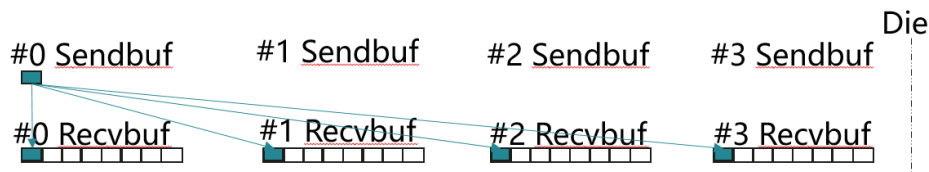
#### (1) 算法实现

Allgather 算子优化收益主要来源于基于拓扑感知的树形算法和向量化的数据搬运，算法实现分为四个步骤，其中 Step 2 和 Step 3 并行执行：

Step 1. 头同步，保证可读取其他进程的 Sendbuf 数据。

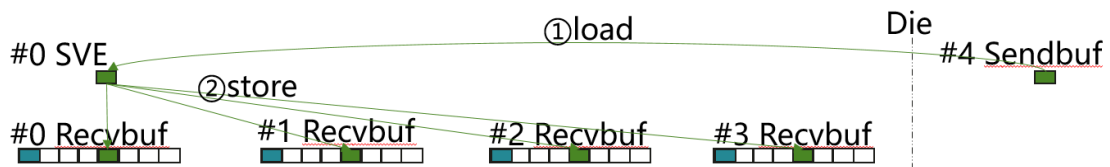
Step 2. Die 内 Scatter。

进程 0 将自己 Sendbuf 中的数据写到所在 Die 内 4 个进程的 Recvbuf 中。



Step 3. 跨 Die 读，再做 Die 内 Scatter。

进程 0 读取进程 4 的 Sendbuf，并将读到的数据写到所在 Die 内 4 个进程的 Recvbuf 中。此步骤中，通过 SVE 指令将数据加载到 SVE 寄存器，再从寄存器 store 到多块不同的地址空间，相比直接调用 memcpy，将 load 次数减少到原来的四分之一。



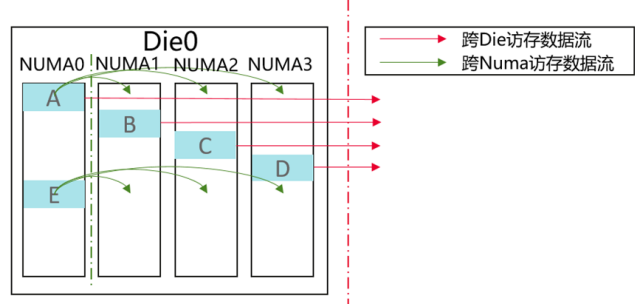
Step 4. 尾同步，保证所有进程 store 操作完成，数据可用。

#### (2) 算子性能

在表 1-3 中，我们列出了不同 size 下单算子实测性能，并展示了算子内各阶段性能拆解。在 Step2、3 并行执行的过程中，同 Die 上的四个进程分别跨 Die 读取了一倍 sendbuf 的数据，即对于每个 Die 而言的跨 Die 读取数量为  $4 \times \text{Sendbuf}$ ；同时，每个 NUMA 向同 Die 的其他进程共写入六倍 sendbuf 数据，



即对于单个 NUMA 而言的读写数量为  $6 \times \text{Sendbuf}$ ，数据流向如图所示：



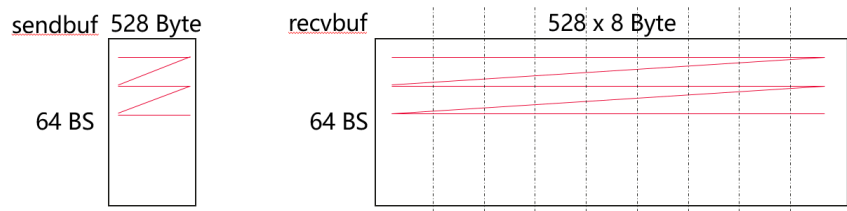
结合访存数据量和该步骤的时延，我们计算了跨 Die 访存和跨 NUMA 访存的带宽，结合 1.3.3 节数据给出了带宽利用率。

|                    |              |       |        |        |        |        |        |
|--------------------|--------------|-------|--------|--------|--------|--------|--------|
| 输入 size (KByte)    |              | 112   | 224    | 448    | 896    | 1344   | 1792   |
| 单算子时延(us)          |              | 17.43 | 28.21  | 34.97  | 46.66  | 81.13  | 108.53 |
| 算子拆解时延<br>(us)     | fence        | 1.49  | 1.51   | 1.50   | 1.32   | 1.35   | 1.35   |
|                    | parallel in  | 1.16  | 1.26   | 1.32   | 1.44   | 1.61   | 1.64   |
|                    | step2+3 并行   | 8.40  | 10.52  | 20.72  | 26.65  | 39.98  | 48.63  |
|                    | parallel out | 3.56  | 10.57  | 6.45   | 8.38   | 7.25   | 10.70  |
|                    | fence        | 2.21  | 3.79   | 4.42   | 8.29   | 30.34  | 45.65  |
| 跨 Die 单向读带宽 (GB/s) |              | 50.83 | 81.26  | 82.47  | 128.25 | 128.23 | 140.56 |
| 跨 NUMA 读写带宽 (GB/s) |              | 76.25 | 121.89 | 123.71 | 192.38 | 192.34 | 210.84 |
| 跨 Die 单向读带宽利用率     |              | 30%   | 48%    | 49%    | 75%    | 75%    | 83%    |
| 跨 NUMA 读写带宽利用率     |              | 31%   | 49%    | 49%    | 77%    | 77%    | 84%    |

#### 1.4.4 Batch\_Allgather

##### (1) 算法实现

相比标准语义的 Allgather, Batch\_Allgather 类似于矩阵拼接操作, 以 TP8、BS64 场景为例, 如图所示为 Sendbuf 和 Recvbuf 的目标形状及内存序, 对于每一行可看作标准语义的 Allgather。



其优化思路与标准语义 Allgather 基本相同, 区别在于: 标准语义 Allgather 多线程直接按内存序切分 Sendbuf, 而 Batch\_Allgather 按线程数切分 Batch Size 维度, 以 32 线程, 输入 BS64 为例, 每个线程处理 2 个 Batch Size, 即每个线程做两次标准语义的 Allgather (算法实现同上节)。

## (2) 算子性能

在下表中，我们列出了不同 size 下单算子实测性能，并展示了算子内各阶段性能拆解。在 Step2、3 并行执行过程中，跨 Die 和跨 NUMA 访存数据量与上节分析相同，结合访存数据量和该步骤的时延，我们计算了跨 Die 访存和跨 NUMA 访存的带宽，结合 1.3.3 节数据给出了带宽利用率。

|                   |              |        |         |         |         |         |          |
|-------------------|--------------|--------|---------|---------|---------|---------|----------|
| 输入包大小(Byte)       |              | 64*582 | 64*1024 | 64*2048 | 64*4096 | 64*8192 | 64*16384 |
| 单算子时延(us)         |              | 14.23  | 15.83   | 20.15   | 30.94   | 41.91   | 74.5     |
| 算子拆解<br>时延(us)    | fence        | 2.56   | 2.98    | 2.49    | 1.90    | 1.96    | 1.94     |
|                   | parallel in  | 1.02   | 0.88    | 0.70    | 1.26    | 1.18    | 1.37     |
|                   | step2+3 并行   | 5.24   | 7.20    | 11.77   | 19.70   | 27.46   | 41.75    |
|                   | parallel out | 3.17   | 2.44    | 2.79    | 4.85    | 8.77    | 17.32    |
|                   | fence        | 1.58   | 1.68    | 1.74    | 2.11    | 1.86    | 11.50    |
| 跨 Die 单向读带宽(GB/s) |              | 26.47  | 33.89   | 41.48   | 49.57   | 71.13   | 93.56    |
| 跨 NUMA 读写带宽(GB/s) |              | 39.71  | 50.84   | 62.22   | 74.37   | 106.70  | 140.34   |
| 跨 Die 单向读带宽利用率    |              | 16%    | 20%     | 24%     | 29%     | 42%     | 55%      |
| 跨 NUMA 读写带宽利用率    |              | 16%    | 20%     | 25%     | 30%     | 43%     | 56%      |